

Update the EXISTING SaveStreet Dogs website. Do not rebuild the entire website from scratch.

The current website already has a good visual identity, homepage sections, navigation, puppies, rescue, adoption, shop/products, stories, learn, donate, foster and volunteer sections.

Keep the existing design direction, but now turn it into a ****proper functional production-ready website****.

The current website looks visually good but still feels like a demo organization website in several places. Improve the functionality, navigation, authentication, admin system, product ordering, and overall UX.

1. MOST IMPORTANT: PROPER AUTHENTICATION ■

Create a real authentication system.

Users should be able to:

- Sign Up
- Sign In
- Sign Out
- Reset/Forgot Password
- Stay signed in across sessions
- View their account
- View their orders
- View their submitted rescue/adoption/volunteer/foster requests where applicable

Do NOT create fake login/logout buttons.

The authentication must actually work.

Use the project's existing authentication/backend architecture if one already exists.

If Firebase/Supabase/authentication is already configured, use the existing setup instead of creating a second authentication system.

2. USER ACCOUNT

Create:

`/account`

After login, the user should be able to see:

Profile

- Name
- Email
- Phone
- Address

- Account information

My Orders

Show:

- Order ID
- Product
- Quantity
- Amount
- Payment status
- Order status
- Expected delivery
- Order date

My Activity

Where applicable:

- Rescue reports
- Adoption applications
- Foster applications
- Volunteer applications

Do not expose other users' information.

3. SIGN IN / SIGN UP UI

Create beautiful dedicated pages:

`/login`

`/signup`

`/forgot-password`

The design must match SaveStreet Dogs.

Include proper:

- Validation
- Error messages
- Loading states
- Success messages
- Password visibility toggle
- Disabled/loading submit buttons

Example:

****Signing in...****

instead of allowing repeated submissions.

4. NAVIGATION AUTH STATE

The navbar should dynamically change based on authentication.

When logged out:

****Sign In****

****Create Account****

When logged in:

Show a small account/profile icon or:

****My Account****

and provide:

****Sign Out****

Sign Out must actually terminate the current authenticated session.

After signing out:

- Clear authenticated user state
- Return the user to the public website
- Prevent access to protected pages
- Do not leave stale account information visible

5. ADMIN AUTHENTICATION ■

Create a completely separate protected admin area:

``/admin``

The admin panel **MUST NOT** be publicly accessible.

Do not simply hide the admin link.

Implement actual authorization.

There should be a distinction between:

Normal User

Can:

- Browse
- Adopt/apply
- Report
- Buy products
- Donate
- Volunteer
- Foster
- View own account/orders

Admin

Can:

- Manage products
- Manage orders
- Manage users
- Manage rescue reports
- Manage adoption applications
- Manage foster applications
- Manage volunteers
- Manage stories
- Manage puppies/dogs
- Manage delivery
- Update order statuses

A normal authenticated user must NOT be able to access admin functions by manually typing `/admin`.

6. ADMIN LOGIN

Create a proper admin authentication flow.

Admin should sign in securely.

Do NOT hardcode an admin password in frontend code.

Do NOT put credentials inside:

- React components
- JavaScript files
- `vercel.json`
- GitHub repository
- public environment variables

Use secure environment variables/backend authentication.

7. ADMIN DASHBOARD

Create a polished dashboard.

Sidebar:

- Dashboard
- Orders
- Products
- Puppies/Dogs
- Rescue Reports
- Adoption Applications
- Foster Applications
- Volunteers
- Customers
- Donations
- Stories
- Settings

Dashboard overview cards:

- Total Orders
- Pending Orders
- Products
- Rescue Reports
- Adoption Applications
- Volunteer Applications

Use real database values when available.

Do not create fake statistics and present them as real.

8. PRODUCT MANAGEMENT ■■

The admin should be able to:

- Add product
- Upload product image
- Edit product
- Delete/archive product
- Change price
- Change stock
- Change availability

- Add description
- Add category
- Add multiple images

Current product requirement:

****60 per piece****

Availability:

****Deliverable across India****

Show:

****Delivery charges may apply depending on location.****

Do not mention any internal location or warehouse location publicly.

9. PRODUCT ORDERING SYSTEM

The public `/products` section should be a real shopping experience.

Customer:

1. Opens Products
2. Selects product
3. Views product details
4. Selects quantity
5. Adds to cart
6. Opens cart
7. Proceeds to checkout
8. Enters delivery details
9. Pays
10. Order is created
11. Customer receives Order ID
12. Order becomes visible in Admin Panel

10. CHECKOUT

Collect:

- Full Name
- Phone Number
- Email
- Full Address

- City
- State
- PIN Code
- Quantity
- Delivery notes (optional)

Validate all required fields.

The customer should clearly see:

- Product price
- Quantity
- Subtotal
- Delivery charge if applicable
- Final total

Do NOT make up a delivery charge.

If delivery calculation is not implemented yet, show:

****Delivery charges will be confirmed based on delivery location.****

11. RAZORPAY PAYMENT ■

Prepare the checkout system for Razorpay.

Payment flow:

Customer → Checkout → Razorpay → Payment verification → Order creation/confirmation.

Use environment variables for Razorpay credentials.

Never expose secret keys in frontend code.

Do NOT fake successful payments.

If Razorpay is not connected yet, create a clearly marked integration-ready payment layer rather than pretending the payment succeeded.

Store:

- Payment ID
- Payment status
- Order ID
- Amount
- Payment timestamp

where securely available.

12. ADMIN ORDERS ■

Create a powerful Orders page.

Each order should show:

- Order ID
- Customer name
- Phone number
- Email
- Full address
- Product
- Quantity
- Product price
- Delivery charge
- Total amount
- Payment status
- Order date
- Expected delivery
- Current order status

Admin can open an order to see all details.

13. ORDER STATUS WORKFLOW

Create a proper order-status system.

Statuses:

1. Payment Pending
2. Payment Verified
3. Preparing
4. Ready to Dispatch
5. Dispatched
6. Out for Delivery
7. Delivered
8. Delayed
9. Cancelled

Admin should be able to manually update the status.

When status changes, save it to the database.

Do not only change the UI locally.

14. EXPECTED DELIVERY

Default expected delivery:

****1–2 days after successful order placement****

For example:

Order on 10th → expected around 11th–12th.

However, the admin must be able to manually modify the expected delivery date.

If an order is delayed:

Admin selects:

****Delayed****

and can enter:

****Delay reason****

The customer should then see:

****Order Delayed****

along with the updated expected delivery date/reason where appropriate.

15. MANUAL DELIVERY ASSIGNMENT

Do NOT integrate a delivery partner API yet.

Create the structure for future use.

Admin can manually assign delivery:

- Delivery partner name
- Phone number
- Tracking ID
- Notes

For now this is manually entered by the admin.

Keep the architecture ready for future delivery integration.

16. CUSTOMER ORDER TRACKING

Create:

`/orders/:orderId`

Customer can see their order timeline:

****Payment → Preparing → Ready to Dispatch → Dispatched → Out for Delivery → Delivered****

Show:

- Order ID
- Product
- Quantity
- Total
- Current status
- Expected delivery
- Delivery information

Only the owner of the order or authorized admin can access the order.

17. RESCUE REPORTS ■

The existing Report a Dog page should submit actual data.

Save:

- Reporter name
- Phone/email
- Location
- Dog condition
- Description
- Urgency
- Photo
- Submission time
- Status

Admin can change:

- New
- Reviewing
- Assigned
- In Progress
- Resolved
- Closed

Admin can add internal notes.

Protect private reporter information.

18. PUPPIES / DOG MANAGEMENT

Admin should be able to:

- Add puppy/dog
- Upload images
- Edit profile
- Change adoption status
- Archive profile
- Update location/general information
- Add rescue story

Statuses:

- Available
- Under Care
- Foster Needed
- Adoption Pending
- Adopted

The public Puppies page should automatically reflect admin updates.

19. ADOPTION APPLICATIONS

When someone submits an adoption application:

Save it to the database.

Admin should see:

- Applicant
- Contact
- Application date
- Dog/puppy
- Status
- Notes

Status options:

- New
- Under Review
- Contacted
- Approved
- Not Approved
- Completed

Do not expose applicant information publicly.

20. VOLUNTEER + FOSTER

Make the existing forms functional.

Save submissions to the admin panel.

Admin can review and update statuses.

21. CURRENT WEBSITE CONTENT IMPROVEMENTS

The current homepage contains phrases such as:

"Demo organization content."

Do not display this prominently on the public website if the content is being presented as the real SaveStreet Dogs website.

However, do NOT fabricate real-world claims.

If a story, statistic, testimonial, rescue, partnership, certification, or impact number is only sample content, either:

- Clearly mark it as sample content in the appropriate internal/demo environment, or
- Replace it with neutral non-factual copy that does not pretend the event actually happened.

Do not invent rescue statistics.

22. CURRENT HERO / BRANDING

Keep the existing core identity:

****Street by street. Paw by paw. Saving lives.****

Keep:

****Every Paw Deserves a Chance.****

The current design direction is good.

Improve it rather than destroying it.

Keep the website:

- Clean
- Emotional but not manipulative
- Premium
- Modern

- Trustworthy
- Animal-focused

23. MOBILE NAVIGATION ■

Make sure the three-line hamburger menu is clearly visible on mobile.

Desktop:

Full navigation.

Mobile:

Logo + hamburger.

The hamburger must actually open the menu.

Mobile shortcuts should make it very easy to find:

- ■ Puppies
- ■ Report a Dog
- ■ Adoption
- ■■ Shop
- ■ Donate

This is a priority.

24. MOBILE RESPONSIVENESS

Test the entire website at:

320px

360px

375px

390px

414px

768px

1024px

1280px+

Fix:

- Overflow
- Broken grids
- Text clipping
- Buttons

- Forms
- Images
- Header
- Footer
- Admin tables
- Product cards
- Checkout
- Cart
- Modals

No unnecessary horizontal scrolling.

25. BUTTON AUDIT

Every button must perform its intended action.

Remove/fix dead buttons.

Test:

- Report a dog
- Find a puppy
- Adoption
- Shop
- Stories
- Learn
- Donate
- Volunteer
- Foster
- Contact
- Add to cart
- Checkout
- Login
- Signup
- Logout
- Submit forms
- Admin controls
- Order status controls

Do not leave placeholder buttons that do nothing.

26. CART

Create a persistent cart.

Allow:

- Add product
- Remove product
- Increase quantity
- Decrease quantity
- View subtotal
- Checkout

Do not allow quantity below 1.

Handle empty cart properly.

27. SECURITY

Implement proper security.

Important:

- Protected admin routes
- Server-side authorization
- Input validation
- Secure authentication
- Secure file uploads
- Environment variables
- No secret keys in frontend
- No passwords stored as plain text
- No sensitive customer data exposed publicly
- Validate ownership before displaying orders

28. DATABASE

Use the existing database architecture if available.

Ensure proper models/collections exist for:

- Users
- Products
- Orders
- OrderItems
- Payments
- Delivery

- Puppies/Dogs
- RescueReports
- AdoptionApplications
- FosterApplications
- VolunteerApplications
- Stories

Use proper relationships.

29. ADMIN RESPONSIVE DESIGN

The Admin Panel must ALSO be responsive.

On mobile:

- Sidebar becomes a hamburger/drawer
- Tables become responsive cards or horizontally scrollable only where necessary
- Order details remain readable
- Status controls remain accessible
- Forms fit the screen

Do not create a desktop-only admin panel.

30. FINAL QA

Before finishing, test the entire website as three different users:

Visitor

Can browse everything publicly.

Customer

Can sign up → sign in → buy product → pay → view order → sign out.

Admin

Can sign in → view order → verify payment → update status → assign delivery manually → mark delayed/delivered.

Test:

- Refreshing pages
- Direct URLs
- Mobile navigation
- Authentication persistence

- Logout
- Protected routes
- Admin authorization
- Cart
- Checkout
- Forms
- Order creation
- Order tracking
- Product management

Fix all console errors and broken interactions.

The final result should no longer feel like a static demo website.

It should feel like a **real SaveStreet Dogs platform** with:

****Public Website + User Accounts + Product Store + Ordering + Payments Architecture + Order Tracking + Admin Dashboard + Rescue/Adoption Management.****

Preserve the existing beautiful SaveStreet Dogs design while making the entire system actually functional.